DOCUMENTAÇÃO TÉCNICA & MANUAL ACADÊMICO

SISTEMA FAZ IT

E-Commerce, Controle de Produção e Gestão Logística para Circuitos Cartões de Alimentos

Versão de Estudo

Integralmente Comentada

Arquitetura

📚 Guia de Estudo e Roteiro de Aprendizagem — Clube Faz Bem V2

Bem-vindo à **Versão de Estudo do Clube Faz Bem V2**. Este material foi estruturado para fornecer uma compreensão profunda e detalhada de toda a arquitetura, regras de negócio, modelagem de dados, segurança e implementação prática do sistema.

🗺️ Mapa dos Documentos de Estudo

Para aproveitar ao máximo este material, recomendamos seguir a trilha de aprendizado abaixo:

Documento	Descrição	Foco Principal
01. Visão Geral e Objetivo	O que é o projeto, qual problema resolve e suas regras de negócio.	Negócio, Personas, Ciclo Semanal
02. Arquitetura do Sistema	Como o sistema está organizado, fluxo de requisições e design patterns.	MVC/REST, Ciclo de Requisição, Sessão
03. Banco de Dados e Modelos	Estrutura de tabelas, relacionamentos e a camada `app/Models/`.	SQL, Entidades, Métodos POO
04. Backend: APIs e Fluxos Operacionais	Detalhamento de todos os endpoints RESTful, parâmetros e webhooks.	Rotas, Lógica de Negócio, Mercado Pago
05. Frontend e Interfaces	Estrutura das telas, manipulação de DOM, JavaScript assíncrono e CSS.	HTML5, CSS3, JS Vanilla, SPA/MPA
06. Guia de Segurança e Boas Práticas	Como o sistema se protege contra ataques (CSRF, XSS, SQLi, Brute Force).	Segurança Defensiva, OWASP, RBAC
07. Guia de Conteúdos e Conceitos de Programação	Catálogo de conteúdos de computação (Vetores, POO, SQL, Algoritmos, HTTP, etc).	Conceitos Acadêmicos, Estruturas, Desafios

📂 Estrutura de Diretórios Desta Pasta de Estudo

```
versao_estudo/
│
├── 00_INDICE_E_GUIA_DE_ESTUDO.md         # Este guia de navegação
├── 01_VISAO_GERAL_E_OBJETIVO.md          # Documentação de Produto & Negócio
├── 02_ARQUITETURA_DO_SISTEMA.md          # Arquitetura de Software & Design
├── 03_BANCO_DE_DADOS_E_MODELOS.md        # Modelagem de Dados & Camada Model
├── 04_BACKEND_APIS_E_FLUXOS.md           # APIs Backend & Integrações
├── 05_FRONTEND_E_INTERFACES.md           # Camada de Interface & Scripts JS
├── 06_GUIA_DE_SEGURANCA_E_BOAS_PRATICAS.md # Hardening, Segurança & RBAC
├── 07_GUIA_DE_CONTEUDOS_E_CONCEITOS.md   # Catálogo Temático de Conteúdos (POO, Vetores, etc)
│
├── app/                                  # Núcleo POO do Backend (100% Comentado)
│   ├── Database.php                      # Conexão PDO com padrão Singleton
│   ├── Env.php                           # Parser manual de arquivos .env
│   ├── Security.php                      # Proteção CSRF, Rate Limiting e Cookies
│   ├── Validator.php                     # Validações estruturadas (CPF, etc)
│   ├── MercadoPagoService.php            # Integração cURL com Checkout Transparente
│   └── Models/                           # Classes de Entidade e Regra de Negócio
│       ├── Assinatura.php                # Gerenciamento de assinaturas
│       ├── Cliente.php                   # Dados e histórico de clientes
│       ├── Dashboard.php                 # Agregação de métricas financeiras/operacionais
│       ├── Pedido.php                    # Processamento de carrinho e persistência
│       ├── Preferencia.php               # Restrições alimentares do cliente
│       ├── Producao.php                  # Kit da semana e fechamento de ciclo
│       ├── Produto.php                   # Catálogo e controle de inventário
│       └── Usuario.php                   # Autenticação, perfis e equipe
│
├── apis/                                 # Endpoints RESTful comentados didaticamente
│   ├── config.php                        # Configuração global
│   ├── cors.php                          # Headers HTTP e controle CORS
│   ├── api_csrf.php                      # Emissão de token CSRF
│   ├── api_sessao_v2.php                 # Verificação da sessão do usuário
│   ├── api_cadastro_v2.php               # Cadastro e geração de assinatura
│   ├── api_login_v2.php                  # Login com hash de senha
│   ├── api_recuperar_senha.php           # Redefinição de credenciais
│   ├── api_catalogo_v2.php               # Vitrine de produtos
│   ├── api_checkout_v2.php               # Fechamento de compras extras
│   ├── api_meus_pedidos_v2.php           # Histórico de compras do cliente
│   ├── api_minha_assinatura_v2.php       # Consulta do plano ativo
│   ├── api_gerenciar_assinatura_v2.php   # Pausa, troca de preferências e cancelamento
│   ├── api_faturamento_v2.php            # Gestão e pagamento de faturas mensais
│   ├── api_perfil_v2.php                 # Edição de dados do usuário
│   ├── api_mp_key.php                    # Disponibilização da chave pública MP
│   ├── api_mp_webhook.php                # Webhook assíncrono de pagamentos
│   ├── api_producao_v2.php               # Consulta do kit semanal atual
│   ├── api_separacao_v2.php              # Painel de conferência e pesagem no galpão
│   ├── api_logistica_v2.php              # Roteirização e confirmação pelo entregador
│   ├── api_estoque_v2.php                # Ajuste de inventário
│   ├── api_admin_dashboard_v2.php        # Métricas gerenciais
│   ├── api_admin_pedidos_v2.php          # Gerenciamento de pedidos
│   ├── api_admin_produtos_v2.php         # Cadastro/Edição de produtos
│   ├── api_admin_clientes_v2.php         # Cadastro/Edição de clientes
│   ├── api_admin_funcionarios_v2.php     # Gestão da equipe interna
│   ├── api_admin_editar_kit_semana.php   # Montagem do kit hortifrúti semanal
│   ├── api_admin_gerar_pedidos_v2.php    # Geração em lote dos pedidos do ciclo
│   ├── api_admin_ordem_rotas.php         # Ordenação de rotas logísticas
```

```
│   ├── api_admin_relatorio_compras.php   # Necessidade total de compra no produtor
│   ├── comprovante.php                   # Emissão de recibo/fatura formatado
│   └── logout.php                        # Destruição segura de sessão
│
├── html/                                 # Telas HTML5 estruturadas e comentadas
│   ├── index.html                        # Landing page e apresentação acadêmica
│   ├── catalogo.html                     # Catálogo de produtos e modal de peso
│   ├── carrinho.html                     # Carrinho e checkout de extras
│   ├── cadastro.html                     # Onboarding com geolocalização Leaflet
│   ├── login.html                        # Autenticação e recuperação de senha
│   ├── perfil.html                       # Portal do cliente e pagamento com MP Brick
│   ├── admin.html                        # Painel de controle administrativo SPA
│   ├── separador.html                    # Painel touch para pesagem no galpão
│   ├── entregador.html                   # Painel mobile de rotas com GPS/WhatsApp
│   └── vitrine.html                      # Vitrine pública do kit semanal
│
├── css/                                  # Folhas de estilo CSS3 comentadas
│   ├── index.css, admin.css, ...         # Tokens de design, responsividade e layout
│
└── js/                                   # Scripts JavaScript comentados didaticamente
    ├── utils.js                          # Utilitários globais (Fetch com CSRF, Toasts)
    ├── index.js, catalogo.js, ...        # Módulos interativos de cada tela
```

🎯 Dicas de Estudo e Roteiros Práticos

1. **Entenda o Domínio:** Comece lendo `01_VISAO_GERAL_E_OBJETIVO.md` para compreender a lógica de negócio (ciclo semanal de colheita, kits fixos + extras).
2. **Entenda a Fundação:** Estude `app/Database.php` e `app/Security.php`. Eles sustentam todas as requisições do sistema.
3. **Acompanhe um Fluxo Real:**
4. **Fluxo 1 (Compra):** `cadastro.html` ➡️ `js/cadastro.js` ➡️ `api_cadastro_v2.php` ➡️ `app/Models/Usuario.php`.
5. **Fluxo 2 (Operação Semanal):** `admin.html` ➡️ `api_admin_gerar_pedidos_v2.php` ➡️ `separador.html` ➡️ `api_separacao_v2.php` ➡️ `entregador.html` ➡️ `api_logistica_v2.php`.
6. **Fluxo 3 (Pagamento):** `perfil.html` ➡️ `js/perfil.js` ➡️ `api_faturamento_v2.php` ➡️ `app/MercadoPagoService.php` ➡️ `api_mp_webhook.php`.

🌿 01. Visão Geral e Objetivo do Projeto

📌 Introdução e Propósito

O **Clube Faz Bem** é uma plataforma integrada de e-commerce e gestão logística (ERP leve) desenvolvida especificamente para o modelo de **clube de assinatura de cestas de hortifrúti orgânicos e agroecológicos**.

O objetivo central do projeto é conectar pequenos agricultores locais aos consumidores finais urbanos, eliminando intermediários, reduzindo o desperdício de alimentos perecíveis e garantindo previsibilidade de colheita e receita para o produtor rural.

🎯 O Problema que o Projeto Resolve

No comércio tradicional de hortifrúti fresco:
1. **Desperdício elevado:** Supermercados e feiras perdem até 40% do alimento fresco que não é vendido a tempo.
2. **Incerteza para o agricultor:** O produtor colhe sem saber exatamente quanto conseguirá vender.
3. **Complexidade logística:** Produtos perecíveis exigem colheita sob demanda, montagem rápida de kits personalizados e entregas pontuais em rotas otimizadas.
4. **Cobrança fragmentada:** Cobrar entregas semanais avulsas gera alto custo de transação e atrito para o cliente.

Como o Clube Faz Bem soluciona isso:

- **Modelo de Assinatura com Faturamento Mensal:** O cliente assina um plano (Semanal ou Quinzenal) e paga uma fatura consolidada no início do ciclo mensal.
- **Janela Estrita de Catálogo:** A loja para compra de itens extras fica aberta apenas no início da semana (Domingo à noite até Terça ao meio-dia). Quando o limite de capacidade (200 kits) é atingido, o catálogo fecha automaticamente.
- **Colheita Sob Demanda:** Ao fechar o catálogo na terça-feira, o sistema gera o **Relatório de Compras/Colheita** consolidado, informando ao produtor exatamente a quantidade de pés de alface, maços de rúcula e quilos de tomate a serem colhidos.
- **Fluxo Operacional Sem Papel:** O sistema provê interfaces web responsivas para a equipe do galpão (Separador) e para os motoristas (Entregador), com conferência de peso e roteirização GPS.

👥 Personas do Sistema e Papéis (RBAC)

O sistema possui 4 níveis de usuários com permissões específicas:

```
graph TD
    A[Usuários do Sistema] --> B[Cliente / Assinante]
    A --> C[Administrador / Gestor]
    A --> D[Separador / Galpão]
    A --> E[Entregador / Motorista]

    B --> B1[Escolhe plano, personaliza preferências, compra extras, paga faturas]
    C --> C1[Dashboard financeiro, edita kit semanal, cadastra produtos, gera lotes de pedido]
    D --> D1[Conferência touch dos itens da cesta, ajuste de peso real de itens fracionados]
    E --> E1[Acesso à lista de entregas em ordem, navegação GPS, disparo de WhatsApp, baixa]
```

1. Cliente (Assinante)

- **Cadastro e Escolha de Plano:** Seleciona frequência (Semanal ou Quinzenal), cadastra endereço de entrega e restrições alimentares (ex: não come coentro, beterraba).
- **Área do Cliente (`perfil.html`):**
- Consulta o kit que receberá na semana.
- Pausa a assinatura temporariamente (com concessão automática de créditos para compensar na próxima fatura).
- Consulta e paga suas faturas mensais via Mercado Pago (Pix ou Cartão).
- Histórico detalhado de pedidos anteriores.
- **Catálogo de Extras (`catalogo.html`):** Adiciona itens avulsos à sua entrega semanal caso o catálogo esteja no período de abertura.

2. Administrador / Gestor (`admin.html`)

- **Painel Financeiro e KPIs:** Visualiza faturamento recebido, receita pendente/esperada no mês, quantidade de clientes ativos, total de pedidos e itens em estoque crítico.
- **Configuração do Kit da Semana:** Seleciona quais itens da colheita farão parte da cesta padrão de todos os assinantes naquele ciclo.
- **Geração Automática de Pedidos do Ciclo:** Botão de 1-clique que cria todos os pedidos da semana para os clientes com assinatura ativa, anexando automaticamente os itens do kit padrão e respeitando as trocas e restrições de cada cliente.
- **Relatório de Compras:** Tabela sumarizada de quantos quilos, maços ou unidades de cada legume/verdura precisam ser encomendados aos produtores.
- **Gestão de Produtos, Clientes e Equipe:** CRUD completo com controle de fotos, preços, tipo de venda (Inteiro vs Fracionado/Kg) e controle de estoque.

3. Separador / Estoquista (`separador.html`)

- Interface touch simplificada para operação em tablets ou celulares no galpão.
- Lista os pedidos com status `"Em separação"`.
- Permite abrir a comanda digital de cada cliente, ver as observações e restrições alimentares.
- **Pesagem Real:** Permite ao operador digitar o peso exato de itens pesáveis (ex: o cliente pediu 1kg de batata e o operador pesou 1,050kg), garantindo transparência e integridade de estoque.
- Avança o status do pedido para `"Aguardando Entrega"`.

4. Entregador / Motorista (`entregador.html`)

- Interface touch otimizada para smartphones com suporte a visualização de rotas ordenadas.
- Botão direto para **abrir GPS (Google Maps / Waze)** com o endereço e ponto de referência do cliente.
- Botão de **WhatsApp direto com o cliente** com mensagem pré-formatada avisando sobre a chegada.
- Confirmação de entrega em 1 toque, atualizando o status para `"Entregue"` e gravando o timestamp da entrega.

🔄 O Ciclo Operacional Semanal

O negócio opera em um ciclo semanal bem definido:

```
sequenceDiagram
    autonumber
    actor Admin as Administrador
    actor Cliente as Assinante
    actor Separador as Galpão / Separador
    actor Entregador as Logística / Entregador

    Note over Admin,Cliente: Domingo (18h) a Terça (12h): Janela de Vendas
    Admin->>Admin: Define e publica os itens do Kit da Semana
    Cliente->>Cliente: Acessa catálogo, seleciona itens extras e fecha carrinho

    Note over Admin,Separador: Terça (12h01) a Quarta: Fechamento & Colheita
    Admin->>Admin: Catálogo fecha automaticamente (ou se bater 200 kits)
    Admin->>Admin: Clica em "Gerar Pedidos da Semana"
    Admin->>Admin: Emite Relatório de Compras e envia para os agricultores

    Note over Separador,Entregador: Quinta e Sexta: Montagem & Separação
    Separador->>Separador: Recebe caixas do campo, monta cestas na tela touch e pesa itens
    Separador->>Separador: Marca pedidos como "Aguardando Entrega"

    Note over Entregador,Cliente: Sábado pela manhã: Entregas
    Entregador->>Entregador: Carrega veículo seguindo a ordem da rota
    Entregador->>Cliente: Entrega na residência e confirma entrega no app
```

💳 Modelo de Faturamento e Cobrança

O sistema adota uma abordagem híbrida inteligente para faturamento:

1. **Mensalidade da Assinatura:**
2. No momento do cadastro, o cliente escolhe a frequência (ex: Quinzenal por R$ 50,00 ou Semanal por R$ 100,00).
3. O sistema gera automaticamente uma fatura para o mês corrente (`mes_referencia = 'YYYY-MM'`).

4. O cliente pode quitar sua fatura mensal pela aba **Minhas Faturas** no seu perfil via Mercado Pago (Pix ou Cartão).

5. **Cobrança de Produtos Extras:**

6. Quando o cliente realiza compras extras no catálogo durante a semana, o valor pode ser pago na hora via Mercado Pago Checkout Transparente OU acumulado na fatura mensal aberta do cliente.

7. **Compensação por Pausa de Assinatura:**

8. Se o cliente for viajar ou não quiser receber na semana seguinte, ele ativa a **Pausa da Assinatura**.
9. O sistema credita um saldo de compensação no cadastro do cliente (`saldo_compensacao`), que é automaticamente abatido no valor da próxima fatura (`valor_desconto_creditos`), sem gerar atrito financeiro.

🏗️ 02. Arquitetura do Sistema e Padrões de Projeto

📐 Visão Arquitetural

O **Clube Faz Bem V2** adota uma arquitetura em camadas pragmática, desacoplada e de alto desempenho, orientada a APIs RESTful com backend em **PHP Orientado a Objetos** e frontend em **HTML5/CSS3/JavaScript Vanilla** (sem frameworks externos pesados como React ou Laravel), priorizando simplicidade, segurança e facilidade de manutenção.

```
graph TB
    subgraph Frontend [Camada de Apresentação - Browser]
        HTML[Telas HTML5]
        CSS[Estilos CSS3 Modulares]
        JS[JavaScript Vanilla Assíncrono / utils.js]
    end

    subgraph Middleware [Camada de Segurança e Interceptação]
        CORS[CORS Handler - cors.php]
        SEC[Security Service - Security.php]
        CSRF[CSRF Token Verification]
        RATE[Rate Limiting em Banco / IP]
        AUTH[RBAC & Session Manager]
    end

    subgraph API [Camada de Controladores / Endpoints REST]
        EP_AUTH[api_login_v2 / api_cadastro_v2]
        EP_CAT[api_catalogo_v2 / api_checkout_v2]
        EP_CLI[api_perfil_v2 / api_faturamento_v2]
        EP_ADM[api_admin_dashboard_v2 / api_admin_*]
        EP_OPS[api_separacao_v2 / api_logistica_v2]
        EP_WEB[api_mp_webhook.php]
    end

    subgraph Domain [Camada de Domínio / Regras de Negócio]
        MOD_USU[Usuario Model]
        MOD_PED[Pedido Model]
        MOD_ASS[Assinatura Model]
        MOD_PRO[Produto & Producao Model]
        MOD_DAS[Dashboard Model]
        SVC_MP[MercadoPagoService]
    end

    subgraph Data [Camada de Acesso a Dados]
        DB[Database Singleton - PDO Prepared Statements]
        MYSQL[(Banco de Dados MySQL / MariaDB)]
    end

    HTML --> JS
    JS -->|Fetch API + X-CSRF-Token| CORS
    CORS --> SEC
    SEC --> CSRF
    CSRF --> RATE
    RATE --> AUTH
    AUTH --> API
    API --> Domain
    Domain --> SVC_MP
    Domain --> DB
    DB --> MYSQL
```

🔁 Ciclo de Vida de uma Requisição HTTP (Request Lifecycle)

Cada interação do usuário percorre um pipeline rigoroso de segurança e processamento:

```
sequenceDiagram
    autonumber
    actor User as Navegador do Usuário
    participant JS as utils.js (Fetch Wrapper)
    participant SEC as Security.php
    participant API as Endpoint (ex: api_checkout_v2.php)
    participant Model as Model (ex: Pedido.php)
    participant DB as MySQL (via PDO)

    User->>JS: Clica em "Finalizar Pedido"
    JS->>JS: Injeta header 'X-CSRF-Token' e serializa JSON
    JS->>SEC: HTTP POST /api_checkout_v2.php
    SEC->>SEC: Inicia sessão segura (HttpOnly, SameSite, Secure)
    SEC->>SEC: Security::checkCSRF() - Valida hash do token
    SEC->>SEC: Security::checkRateLimit() - Verifica requisições por IP

    alt CSRF Inválido ou Rate Limit Excedido
        SEC-->>User: HTTP 403 / 429 { "success": false, "message": "..." }
    else Validações Aprovadas
        SEC->>API: Prossegue para o Endpoint
        API->>API: Valida autenticação (tipo_usuario === 'cliente')
        API->>API: Verifica janela operacional (catalogoAberto())
        API->>Model: processarCarrinho() e criarPedido()
        Model->>DB: Inicia Transação (beginTransaction)
        Model->>DB: Recalcula preços com base nos produtos do banco
        Model->>DB: Insere registro em `pedidos` e `itens_pedido`
        Model->>DB: Confirma Transação (commit)
        DB-->>Model: Sucesso
        Model-->>API: ID do Pedido Criado
        API-->>JS: HTTP 200 { "success": true, "pedido_id": 123 }
        JS-->>User: Exibe Toast de Sucesso e redireciona para confirmação
    end
```

🧩 Padrões de Projeto (Design Patterns) Utilizados

1. Singleton (Conexão com Banco de Dados)

A classe `Database` (`app/Database.php`) garante que exista apenas **uma única conexão PDO aberta** por requisição HTTP, economizando conexões no servidor MySQL e evitando overhead de handshakes repetidos.

```
class Database {
    private static $conexao;
    private function __construct() {} // Impede instanciação direta

    public static function getConexao() {
        if (!isset(self::$conexao)) {
            // Inicializa PDO com charset utf8mb4 e ATTR_EMULATE_PREPARES = false
            self::$conexao = new PDO(...);
        }
        return self::$conexao;
    }
}
```

2. Active Record / Data Mapper Híbrido (Camada Model)

As classes dentro de `app/Models/` isolam totalmente os comandos SQL da lógica de apresentação. Os controladores nas APIs apenas chamam métodos de alto nível como:

- `Pedido::processarCarrinho($carrinho)`
- `Usuario::cadastrarCliente(...)`
- `Producao::obterContagemKitsSemana()`

3. Front Controller / API Gateway Leve (`cors.php` + `Security.php`)

Ao invés de repetir configurações em cada arquivo, todos os endpoints iniciam importando `cors.php` e chamando `Security::checkCSRF()` e `Security::checkRateLimit()`, estabelecendo uma barreira de segurança padronizada.

4. Idempotência e Transações Atômicas (ACID)

Operações financeiras e de criação de pedidos utilizam blocos `beginTransaction()`, `commit()` e `rollBack()` para assegurar que se houver falha ao inserir um item, o pedido inteiro será desfeito, impedindo estados inconsistentes no banco.

🗂️ Estrutura de Camadas e Responsabilidades

Camada	Diretório / Arquivos	Responsabilidade
Apresentação (UI)	`*.html`, `css/*.css`	Marcação semântica, responsividade mobile-first e layout moderno.
Lógica do Cliente	`js/*.js`, `js/utils.js`	Captura de eventos, validação de máscaras, persistência de carrinho no `localStorage`, chamadas assíncronas `fetch()`.
Segurança & Infra	`app/Security.php`, `app/Env.php`, `app/Validator.php`	Proteção contra ataques, leitura de variáveis `.env`, sanitização e validação de documentos.
Controladores (APIs)	`api_*.php`	Recepção do payload JSON, autorização de rota (RBAC), orquestração e resposta HTTP com cabeçalhos adequados.
Modelos de Negócio	`app/Models/*.php`	Regras de cálculo, precificação por peso vs unidade, restrições alimentares e manipulação do banco de dados.
Integração Externa	`app/MercadoPagoService.php`	Comunicação cURL com a API do Mercado Pago para pagamentos via Pix e Cartão de Crédito.
Persistência	`app/Database.php`	Conexão segura MySQL via PDO com `PDO::ERRMODE_EXCEPTION`.

🗄️ 03. Banco de Dados e Modelos de Domínio

📊 Modelo Entidade-Relacionamento (DER)

O banco de dados relacional MySQL/MariaDB do **Clube Faz Bem** é estruturado para suportar o modelo de recorrência, pedidos extras, geolocalização e histórico de pagamentos com integridade referencial.

```
erDiagram
    USUARIOS ||--o{ ENDERECOS : "possui"
    USUARIOS ||--|| ASSINATURAS : "mantém"
    USUARIOS ||--o{ FATURAS_MENSAIS : "possui"
    USUARIOS ||--o{ PEDIDOS : "realiza"
    USUARIOS ||--o{ PREFERENCIAS : "configura"

    PEDIDOS ||--|{ ITENS_PEDIDO : "contém"
    PRODUTOS ||--o{ ITENS_PEDIDO : "composto_por"
    PRODUTOS ||--o{ ITENS_KIT_SEMANA : "selecionado_em"

    USUARIOS {
        int id PK
        string nome
        string email UK
        string senha
        string telefone
        string cpf
        string endereco
        string ponto_referencia
        enum tipo_usuario "cliente, admin, entregador, separador"
        decimal saldo_compensacao
        timestamp criado_em
    }

    ENDERECOS {
        int id PK
        int usuario_id FK
        string logradouro
        string ponto_referencia
        boolean is_principal
        decimal latitude
        decimal longitude
    }

    ASSINATURAS {
        int id PK
        int usuario_id FK
        enum frequencia "Semanal, Quinzenal"
        decimal valor_mensal
        enum status "Ativa, Pausada, Cancelada"
        timestamp ultima_pausa
        timestamp criado_em
        timestamp atualizado_em
    }

    FATURAS_MENSAIS {
        int id PK
        int usuario_id FK
        string mes_referencia "YYYY-MM"
        decimal valor_mensalidade
        decimal valor_extras
        decimal valor_desconto_creditos
        decimal valor_total
        enum status "Pendente, Pago, Cancelado"
        string transacao_id
```

```
        timestamp pago_em
        timestamp criado_em
    }

    PEDIDOS {
        int id PK
        int usuario_id FK
        decimal valor_total
        enum status_pagamento "Pendente, Pago, Cancelado"
        enum status_entrega "Em separacao, Aguardando Entrega, Entregue"
        enum tipo_pedido "Semanal, Extra"
        string forma_pagamento
        string obs_pontual
        string transacao_id
        timestamp data_pedido
        timestamp entregue_em
    }

    ITENS_PEDIDO {
        int id PK
        int pedido_id FK
        int produto_id FK
        decimal quantidade
        decimal quantidade_real
        decimal preco_unitario
        decimal preco_real
    }

    PRODUTOS {
        int id PK
        string nome
        string categoria
        decimal preco
        string unidade "kg, g, un, maco, pe"
        decimal estoque_atual
        decimal peso_estimado_g
        string imagem_url
        enum tipo_venda "Inteiro, Fracionado"
        boolean temporario
        int duracao_dias
    }

    PREFERENCIAS {
        int id PK
        int usuario_id FK
        enum tipo "Troca Fixa, Observacao"
        string descricao
    }

    ITENS_KIT_SEMANA {
        int id PK
        int produto_id FK
        decimal quantidade_padrao
        timestamp criado_em
    }
```

```
    RATE_LIMITS {
        string ip PK
        string endpoint PK
        int attempts
        int first_attempt
    }
```

🏛️ Detalhamento das Classes Model (`app/Models/`)

As classes de domínio encapsulam a regra de negócio e a persistência via PDO:

1. `app/Database.php`

- **Função:** Fornece a conexão única PDO com o banco.
- **Destaques:** Configurado com `PDO::ATTR_EMULATE_PREPARES => false` para prevenir ataques de SQL Injection em segundo plano e `PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION` para lançar exceções capturáveis em falhas de banco.

2. `app/Models/Usuario.php`

- **Função:** Autenticação, criação de clientes e colaboradores.
- **Métodos Principais:**
- `buscarPorEmail($email)`: Localiza o usuário para autenticação com hash `password_verify()`.
- `cadastrarCliente(...)`: Executa uma **transação atômica** que insere o registro na tabela `usuarios`, grava as coordenadas em `enderecos`, cria a `assinatura` inicial e gera a primeira `fatura_mensal`.
- `cadastrarMembroEquipe(...)`: Cadastra administradores, separadores e entregadores.
- `atualizarPerfil(...)`: Permite ao cliente atualizar telefone, endereço e senha.

3. `app/Models/Pedido.php`

- **Função:** Criação, cálculo seguro de carrinho e histórico de pedidos.
- **Métodos Principais:**
- `processarCarrinho($carrinho)`: **Regra de ouro de segurança e precificação.** Ignora os preços enviados pelo JavaScript do navegador, busca os preços reais no banco de dados e recalcula subtotais com base na unidade de venda (unidade ou gramas).
- `criarPedido(...)`: Insere o cabeçalho do pedido e os itens em uma transação PDO. Se o pedido for do tipo 'Extra', ele localiza a fatura mensal pendente do cliente e atualiza o campo `valor_extras` e `valor_total`.
- `atualizarStatus($id, $tipo, $valor)`: Atualiza status de pagamento ou entrega com verificação estrita de whitelist contra SQL Injection.

4. `app/Models/Assinatura.php`

- **Função:** Gerenciamento do ciclo da assinatura do cliente.
- **Métodos Principais:**
- `buscarPorUsuario($usuario_id)`: Retorna os dados do plano, status e data da última pausa.

- `atualizar($usuario_id, $frequencia, $status, $update_pausa)`: Altera a frequência (Semanal/Quinzenal) ou o status (Ativa/Pausada/Cancelada).

5. `app/Models/Cliente.php`

- **Função:** Gestão de clientes para o painel administrativo.
- **Métodos Principais:**
- `listarTodos()`: Executa uma consulta agregada com `LEFT JOIN` e subqueries trazendo o total já gasto pelo cliente e suas preferências alimentares agrupadas com `GROUP_CONCAT`.
- `atualizar(...)`: Atualiza dados cadastrais, endereço e frequência em lote.

6. `app/Models/Producao.php`

- **Função:** Regras do ciclo semanal de produção e colheita.
- **Métodos Principais:**
- `catalogoAberto()`: Valida se a loja de extras pode receber pedidos com base no dia da semana/horário (abre domingo às 18h e fecha terça ao meio-dia) e se o limite de 200 kits foi atingido.
- `gerarRelatorioHortalicas()`: Soma todas as quantidades necessárias de cada hortaliça para os pedidos abertos da semana, formatando a exibição (ex: `12 pés`, `3,5 kg`, `15 maços`).
- `calcularEstoqueDecremento(...)`: Calcula o abatimento exato do inventário convertendo gramas/quilos e unidades.

7. `app/Models/Produto.php`

- **Função:** CRUD de produtos e estoque.
- **Métodos Principais:**
- `listarTodos()`: Retorna todos os produtos cadastrados.
- `salvar(...)` e `atualizar(...)`: Insere e edita informações, preços, imagens e duração de produtos sazonais/temporários.
- `deletar($id)`: Remove um item do catálogo.

8. `app/Models/Preferencia.php`

- **Função:** Registro de restrições alimentares do cliente.
- **Métodos Principais:**
- `adicionar($usuario_id, $tipo, $descricao)`: Grava restrições como "Troca Fixa" (ex: "Não consumo: Pepino") ou observações especiais.
- `buscarPorUsuario($usuario_id)`: Retorna as restrições cadastradas.
- `remover($id, $usuario_id)`: Remove uma preferência com validação de posse (`usuario_id`).

9. `app/Models/Dashboard.php`

- **Função:** Inteligência de negócios e métricas financeiras.
- **Métodos Principais:**
- `getResumoGeral()`: Consolida faturamento pago, faturamento esperado no mês, quantidade de clientes, pedidos totais, alertas de estoque crítico e lista de assinantes inativos.

🔌 04. Backend: APIs RESTful e Fluxos Operacionais

📋 Catálogo Completo de Endpoints da Aplicação

Todas as APIs do sistema comunicam-se exclusivamente em formato **JSON** (`Content-Type: application/json`), utilizam status codes HTTP semânticos (200, 400, 403, 405, 429, 500) e implementam validação de segurança via `Security.php`.

1. Autenticação, Sessão e Segurança

Endpoint	Método	Descrição	Permissão
`api_csrf.php`	`GET`	Retorna o token CSRF da sessão atual para injeção no cabeçalho `X-CSRF-Token`.	Público
`api_sessao_v2.php`	`GET`	Retorna o estado atual da sessão (se está logado, nome, e-mail e cargo do usuário).	Público
`api_cadastro_v2.php`	`POST`	Cria nova conta de cliente, endereço geolocalizado, assinatura e fatura inicial.	Público
`api_login_v2.php`	`POST`	Autentica e-mail e senha com `password_verify`, gerando nova sessão segura e token CSRF.	Público
`api_recuperar_senha.php`	`POST`	Processa solicitações de redefinição de credenciais de acesso.	Público
`logout.php`	`GET/POST`	Destrói a sessão no servidor e remove os cookies de autenticação.	Logado

2. Área do Assinante e Compras Extras

Endpoint	Método	Descrição	Permissão
`api_catalogo_v2.php`	`GET`	Retorna os produtos disponíveis para compra e o status do catálogo (aberto/fechado).	Público
`api_producao_v2.php`	`GET`	Retorna os itens que compõem o **Kit da Semana** e o total de kits encomendados.	Público
`api_checkout_v2.php`	`POST`	Processa pedidos extras da semana com validação de preços no banco e gateway MP.	Cliente
`api_meus_pedidos_v2.php`	`GET`	Lista o histórico de pedidos e entregas do cliente logado.	Cliente
`api_minha_assinatura_v2.php`	`GET`	Retorna a frequência do plano, status e restrições alimentares do cliente.	Cliente
`api_gerenciar_assinatura_v2.php`	`POST`	Permite pausar a assinatura, alterar frequência, gerenciar preferências ou cancelar.	Cliente
`api_faturamento_v2.php`	`GET/POST`	Consulta faturas mensais em aberto e processa pagamentos via Pix / Cartão.	Cliente
`api_perfil_v2.php`	`GET/POST`	Consulta e atualiza dados de cadastro (telefone, endereço, senha).	Cliente

---

3. Integração de Pagamento com Mercado Pago

Endpoint	Método	Descrição	Permissão
`api_mp_key.php`	`GET`	Retorna a chave pública (`MERCADO_PAGO_PUBLIC_KEY`) para o frontend renderizar o Brick.	Público
`api_mp_webhook.php`	`POST/GET`	Recebe as notificações assíncronas do Mercado Pago (IPN) e baixa a fatura como `"Pago"` .	Webhook MP
`comprovante.php`	`GET`	Gera uma via de visualização e impressão de recibo/fatura com QR Code e detalhes.	Logado

---

4. Operações de Galpão e Logística (Mobile)

Endpoint	Método	Descrição	Permissão
`api_separacao_v2.php`	`GET/POST`	**Galpão:** Lista comandas em separação e salva a pesagem real de cada item fracionado.	Separador / Admin
`api_logistica_v2.php`	`GET/POST`	**Entregas:** Retorna rotas de entrega e registra confirmação de entrega com timestamp.	Entregador / Admin

---

5. Gestão Administrativa (`admin.html`)

Endpoint	Método	Descrição	Permissão
`api_admin_dashboard_v2.php`	`GET`	Retorna métricas financeiras (pago, esperado, mensal) e contadores operacionais.	Admin
`api_admin_pedidos_v2.php`	`GET/POST`	Lista e filtra todos os pedidos, permitindo alterar status de pagamento e entrega.	Admin
`api_admin_produtos_v2.php`	`GET/POST/DELETE`	CRUD de itens do catálogo, definição de preços e tipo de venda (Unidade vs Fracionado).	Admin
`api_admin_clientes_v2.php`	`GET/POST`	Lista todos os clientes, histórico de compras, créditos e edição cadastral.	Admin
`api_admin_funcionarios_v2.php`	`GET/POST`	Gerenciamento de colaboradores e atribuição de cargos (`admin` , `separador` , `entregador`).	Admin
`api_admin_editar_kit_semana.php`	`GET/POST`	Seleciona os produtos que farão parte da cesta padrão da semana.	Admin
`api_admin_gerar_pedidos_v2.php`	`POST`	**Lote:** Cria pedidos semanais automaticamente para todos os clientes ativos.	Admin
`api_admin_ordem_rotas.php`	`GET/POST`	Define a sequência de paradas logísticas dos entregadores.	Admin
`api_admin_relatorio_compras.php`	`GET`	Consolida o volume total de compra de cada legume/verdura para o produtor rural.	Admin
`api_estoque_v2.php`	`GET/POST`	Consulta e ajuste manual de saldo em estoque dos produtos.	Admin

---

🔬 Aprofundamento dos Principais Fluxos de Negócio

Fluxo 1: Geração Automática de Pedidos Semanais em Lote (`api_admin_gerar_pedidos_v2.php`)

Na terça-feira à tarde, o administrador clica em **"Gerar Pedidos da Semana"**:

1. O endpoint busca todos os clientes cadastrados com assinatura `status = 'Ativa'`.
2. Para cada cliente, verifica se já não existe um pedido gerado naquela mesma semana (`YEARWEEK`).
3. Busca os itens configurados no **Kit da Semana** padrão.
4. Consulta as **Preferências e Restrições** do cliente (ex: se o cliente marcou que não come "Couve", o sistema substitui ou omite o item).
5. Cria o registro na tabela `pedidos` com `status_entrega = 'Em separação'` e `tipo_pedido = 'Semanal'`.
6. Insere os itens na tabela `itens_pedido`.
7. Conclui em transação segura informando quantos pedidos foram gerados com sucesso.

Fluxo 2: Webhook Assíncrono do Mercado Pago (`api_mp_webhook.php`)

Quando o cliente realiza um pagamento via Pix ou Cartão:

```
sequenceDiagram
    autonumber
    actor MP as Mercado Pago Gateway
    participant WH as api_mp_webhook.php
    participant SVC as MercadoPagoService.php
    participant DB as MySQL Database

    MP->>WH: POST /api_mp_webhook.php?id=123456789&topic=payment
    WH->>SVC: getPayment(123456789)
    SVC->>MP: GET https://api.mercadopago.com/v1/payments/123456789
    MP-->>SVC: Retorna dados completos do pagamento (status, external_reference, etc)
    SVC-->>WH: Payload validado

    alt Pagamento Aprovado (status == 'approved')
        WH->>DB: Atualiza `faturas_mensais` SET status = 'Pago', pago_em = NOW()
        WH->>DB: Atualiza `pedidos` SET status_pagamento = 'Pago'
        WH->>DB: Se assinatura estava pausada/cancelada por falta de pagamento, ativa
        WH-->>MP: HTTP 200 OK
    else Pagamento Pendente ou Rejeitado
        WH->>DB: Registra status correspondente
        WH-->>MP: HTTP 200 OK
    end
```

Fluxo 3: Separação de Pedidos e Ajuste de Pesagem Real (`api_separacao_v2.php`)

1. O separador no galpão visualiza a comanda digital do pedido.
2. Cada produto exibe a quantidade solicitada (ex: `1 kg` de Tomate Italiano).
3. Ao colocar o produto na balança, o operador digita a quantidade real aferida (ex: `1,040 kg`).
4. O endpoint `api_separacao_v2.php` grava `quantidade_real` e recalcula `preco_real` no item do pedido.
5. Quando todos os itens são conferidos, o status do pedido é alterado para `"Aguardando Entrega"`.

🎨 05. Frontend: Interfaces, Scripts e Experiência do Usuário

🌐 Abordagem Técnica do Frontend

O frontend do **Clube Faz Bem** é construído com tecnologias web nativas (**HTML5 Semântico**, **CSS3 Moderno** e **JavaScript ES6+ Vanilla**), dispensando bundlers complexos (como Webpack/Vite) e bibliotecas pesadas. Isso garante:
- **Carregamento instantâneo:** Zero tempo de compilação ou hidratação de framework.
- **Leveza:** Ideal para conexões móveis instáveis dos entregadores e operadores de galpão.
- **Manutenibilidade:** Código legível e compatível diretamente com qualquer navegador moderno.

🛠️ O Núcleo Compartilhado (`js/utils.js`)

O arquivo `js/utils.js` centraliza funções essenciais utilizadas por todas as páginas:

1. `fetchComCSRF(url, options)`: Wrapper sobre a `Fetch API` que busca automaticamente o token CSRF (`api_csrf.php`) e o injeta no cabeçalho `X-CSRF-Token` em todas as requisições `POST`, `PUT` ou `DELETE`.
2. `mostrarToast(mensagem, tipo)`: Sistema de notificações flutuantes (sucesso, erro, alerta) sem dependências externas.
3. `formatarMoeda(valor)`: Formatação monetária padronizada para o padrão brasileiro (`R$ 0,00`).
4. `mascaraTelefone(input)` / `mascaraCPF(input)`: Máscaras dinâmicas de preenchimento para formulários.

📱 Mapeamento de Telas e Módulos JavaScript

```
graph LR
    subgraph Publico [Telas Públicas & Cliente]
        A[index.html] --> A1[index.js]
        B[catalogo.html] --> B1[catalogo.js]
        C[carrinho.html] --> C1[carrinho.js]
        D[cadastro.html] --> D1[cadastro.js]
        E[login.html] --> E1[login.js]
        F[perfil.html] --> F1[perfil.js]
    end

    subgraph Operacional [Telas Operacionais & Gestão]
        G[admin.html] --> G1[admin.js]
        H[separador.html] --> H1[separador.js]
        I[entregador.html] --> I1[entregador.js]
    end
```

1. `index.html` + `js/index.js` (Landing Page Institucional)

- **Objetivo:** Apresentar a proposta de valor do clube, planos de assinatura, funcionamento da colheita e depoimentos.
- **Destaques de Implementação:**
- Carrossel dinâmico e interativo.
- Seção de FAQ expansível com animações suaves em CSS.
- Verificação assíncrona de sessão para alterar botões de "Entrar" para "Minha Conta".

2. `catalogo.html` + `js/catalogo.js` (Catálogo de Produtos Extras)

- **Objetivo:** Permitir aos assinantes adicionar produtos avulsos à entrega semanal.
- **Destaques de Implementação:**
- Banner dinâmico com **contador regressivo** para o fechamento do catálogo.
- Seletor inteligente para produtos fracionados (o cliente escolhe comprar por Unidades estimadas ou por Gramas exatas).
- Cálculo instantâneo do valor estimado no frontend antes de adicionar ao carrinho.

3. `carrinho.html` + `js/carrinho.js` (Checkout e Pagamento)

- **Objetivo:** Revisão dos itens extras selecionados e escolha da forma de pagamento.
- **Destaques de Implementação:**
- Persistência e sincronização de itens no `localStorage`.
- Integração com o SDK do Mercado Pago (**Payment Brick**), permitindo pagamento transparente com cartão de crédito ou chave Pix sem sair do site.
- Opção de adicionar o valor das compras extras à fatura mensal existente.

4. `cadastro.html` + `js/cadastro.js` (Cadastro do Assinante)

- **Objetivo:** Onboarding completo do novo assinante.
- **Destaques de Implementação:**
- Seleção de plano de assinatura (Semanal vs Quinzenal).
- Consulta automática de CEP via API pública com preenchimento automático de logradouro e bairro.
- Captura opcional de coordenadas GPS para roteirização de entregas.
- Seletor de preferências alimentares (exclusões de itens que o cliente não consome).

5. `perfil.html` + `js/perfil.js` (Portal do Cliente)

- **Objetivo:** Central de autoatendimento do cliente.
- **Destaques de Implementação:**
- **Aba Assinatura:** Status da assinatura com botão de 1-clique para Pausar ou Reativar.
- **Aba Faturas:** Lista de faturas com status ("Pago", "Pendente"). Para faturas pendentes, abre o modal de pagamento instantâneo com Mercado Pago.
- **Aba Pedidos:** Histórico de entregas e detalhamento dos itens recebidos.
- **Aba Preferências:** Adição e exclusão de itens que não deseja receber no kit.

6. `admin.html` + `js/admin.js` (Painel Administrativo)

- **Objetivo:** Controle financeiro, operacional e cadastral da cooperativa.
- **Destaques de Implementação:**
- Arquitetura de abas (Tabs) assíncronas para navegação rápida sem recarregar a página.
- Gráficos e cards de KPI (Faturamento Realizado, Faturamento Esperado, Estoque Crítico, Total de Clientes).
- Tabela com filtros dinâmicos de pedidos e exportação/impressão do Relatório de Compras.
- Modais para criação e edição de produtos com upload assíncrono de imagens.

7. `separador.html` + `js/separador.js` (Módulo do Separador / Galpão)

- **Objetivo:** Agilidade e conferência na montagem física das caixas.
- **Destaques de Implementação:**
- Layout com botões grandes, pensado para uso em tablets com luvas ou mãos úmidas.
- Campo de digitação rápida do peso real de itens pesáveis.
- Checklist interativo que valida visualmente quando todos os itens da cesta foram conferidos.

8. `entregador.html` + `js/entregador.js` (Módulo do Entregador)

- **Objetivo:** Apoio ao motorista durante a rota de entregas no sábado.
- **Destaques de Implementação:**
- Cards ordenados pela sequência ideal da rota.
- Botão **"Abrir GPS"**: Inicia a rota no Google Maps ou Waze diretamente no endereço do cliente.
- Botão **"WhatsApp"**: Abre o WhatsApp do cliente com texto padrão informando que o entregador está no portão.

• Botão **"Confirmar Entrega"**: Registra a entrega no sistema em tempo real.

🛡️ 06. Guia de Segurança e Boas Práticas (Hardening)

🔒 Princípios de Segurança Defensiva

O **Clube Faz Bem V2** implementa salvaguardas robustas baseadas nas diretrizes da **OWASP Top 10** e nas melhores práticas de desenvolvimento seguro para proteger dados sensíveis de clientes, transações financeiras e a integridade operacional do negócio.

🛡️ Principais Camadas de Proteção

```
graph TD
    A[Requisição do Cliente] --> B[Proteção de Cookies & Sessão Segura]
    B --> C[Rate Limiting - Anti Força Bruta / DoS]
    C --> D[Validação de Token CSRF]
    D --> E[Controle de Acesso RBAC]
    E --> F[Sanitização e Validação de Inputs]
    F --> G[Prepared Statements Anti-SQLi]
    G --> H[Recálculo Seguro de Preços no Backend]
```

🔐 Detalhamento das Camadas de Segurança

1. Prevenção Contra SQL Injection (SQLi)

- **Prepared Statements Nativos:** Todas as consultas utilizam PDO com parâmetros posicionais (`?`) ou nomeados (`:param`).
- **Desativação de Prepared Emulado:** No arquivo `Database.php`, a diretiva `PDO::ATTR_EMULATE_PREPARES => false` é ativada, instruindo o driver MySQL nativo a tratar dados estritamente como parâmetros, impossibilitando a injeção de comandos SQL mesmo em casos complexos.
- **Whitelists para Campos Dinâmicos:** Quando o nome de uma coluna precisa ser dinâmico (ex: `Pedido::atualizarStatus`), uma lista branca estrita é aplicada para rejeitar qualquer valor não previsto:

```
// app/Models/Pedido.php (SEC-01)
$colunasPermitidas = [
    'pagamento' => 'status_pagamento',
    'entrega'   => 'status_entrega'
];

if (!isset($colunasPermitidas[$tipo])) {
    throw new Exception("Tipo de atualização inválido.");
}

$coluna = $colunasPermitidas[$tipo];
```

> [!IMPORTANT]
> Nunca concatene variáveis de entrada diretamente em strings SQL (`"WHERE id = " . $_GET['id']`). Use sempre `$stmt = $pdo->prepare("WHERE id = ?"); $stmt->execute([$id]);`.

---

2. Proteção Contra CSRF (Cross-Site Request Forgery)

- **Token Criptográfico:** A classe `Security.php` gera tokens aleatórios seguros de 256 bits com `bin2hex(random_bytes(32))`.
- **Comparação em Tempo Constante:** A verificação utiliza `hash_equals()`, impedindo ataques de temporização (*Timing Attacks*).
- **Validação Automática em Métodos de Mutação:** Qualquer requisição `POST`, `PUT` ou `DELETE` deve enviar o cabeçalho `X-CSRF-Token`, caso contrário a requisição é rejeitada com **HTTP 403 Forbidden**.

```
// js/utils.js - Interceptor que injeta o token CSRF automaticamente
const csrfToken = await obterCSRFToken();
headers['X-CSRF-Token'] = csrfToken;
```

---

3. Mitigação de Ataques de Força Bruta e DoS (Rate Limiting)

- O método `Security::checkRateLimit($maxAttempts, $decaySeconds)` limita a quantidade de tentativas por IP em endpoints sensíveis:
- **Login:** Máximo de 10 requisições por minuto.
- **Cadastro:** Máximo de 5 a cada 2 minutos.
- A contagem de requisições é armazenada e limpa na tabela `rate_limits` do MySQL.
- Se o limite for ultrapassado, o servidor responde imediatamente com **HTTP 429 Too Many Requests**.

> [!WARNING]
> O bloqueio por IP previne ataques automatizados de dicionário em endpoints de autenticação e sobrecarga nos servidores compartilhados.

4. Gerenciamento de Sessão e Cookies Seguros

Configurado centralmente na classe `Security.php`:
- **`httponly = true`:** O cookie `PHPSESSID` não pode ser acessado via JavaScript (`document.cookie`), anulando o roubo de sessão via ataques XSS.
- **`samesite = 'Lax'`** / **`'Strict'`:** Impede o envio do cookie de sessão em requisições forjadas vindas de domínios externos.
- **`session_regenerate_id(true)`:** Executado no momento do login para evitar ataques de Fixação de Sessão (*Session Fixation*).

---

5. Criptografia e Armazenamento de Senhas

- As senhas dos usuários **nunca são gravadas em texto puro**.
- O sistema utiliza a função nativa `password_hash($senha, PASSWORD_DEFAULT)`, que aplica o algoritmo **Bcrypt** com fator de custo adaptativo e geração automática de *salt* criptográfico.
- A autenticação é validada exclusivamente com `password_verify($senhaDigitada, $hashSalvo)` na classe `Usuario.php`.

---

6. Controle de Acesso Baseado em Papéis (RBAC) e Prevenção de IDOR

- Cada rota valida explicitamente se o usuário possui a permissão adequada via `cors.php`:
- **Rotas de Administração:** `$_SESSION['tipo_usuario'] === 'admin'`.
- **Rotas de Galpão:** `in_array($_SESSION['tipo_usuario'], ['separador', 'admin'])`.
- **Rotas de Entrega:** `in_array($_SESSION['tipo_usuario'], ['entregador', 'admin'])`.
- **Rotas de Cliente:** `$_SESSION['tipo_usuario'] === 'cliente'` e consultas vinculadas estritamente ao `$_SESSION['usuario_id']` para prevenir **IDOR (*Insecure Direct Object References*)**.

---

7. Integridade Financeira (Recálculo no Servidor)

- O frontend envia apenas os IDs e quantidades dos produtos no carrinho.
- O backend (`Pedido::processarCarrinho`) **ignora completamente qualquer preço ou total enviado pelo cliente** e recalcula todo o subtotal consultando diretamente a tabela `produtos` no banco de dados antes de registrar a cobrança no gateway.

> [!NOTE]
> Essa validação garante que manipulações de valores no DevTools do navegador não alterem o valor real cobrado na fatura.

🧠 Guia Completo de Conteúdos e Conceitos de Programação para Estudo

Este documento cataloga todos os **conceitos fundamentais da Ciência da Computação, Engenharia de Software e Desenvolvimento Web** implementados na prática dentro do **Clube Faz Bem V2**.

Use este guia como um **mapa de estudo temático**, identificando onde cada conceito é aplicado no código-fonte, sua fundamentação teórica e exemplos práticos.

📑 Sumário de Conteúdos

1. Lógica de Programação e Estruturas Fundamentais
2. Vetores, Matrizes e Estruturas de Dados (Arrays & Dicionários)
3. Programação Orientada a Objetos (POO)
4. Padrões de Projeto (Design Patterns) e Arquitetura
5. Banco de Dados Relacional e Linguagem SQL
6. Algoritmos e Lógica Aplicada
7. Desenvolvimento Web, Protocolo HTTP e APIs RESTful
8. Segurança da Informação e Diretrizes OWASP
9. Integrações Externas, Webhooks e APIs de Terceiros
10. Frontend Moderno, UI, CSS e Responsividade

---

1. Lógica de Programação e Estruturas Fundamentais

1.1. Tipos de Dados e Coerção

- **Conceito:** Diferença entre tipos primitivos (inteiros, floats/decimais, strings, booleanos, nulos).
- **No Projeto:**
- Conversão de strings monetárias em floats: `str_replace(',', '.', $_POST['preco'])` em `api_admin_produtos_v2.php`.
- Coerção explícita de tipos: `(int)$_POST['temporario']`, `floatval($item['quantidade'])`.

1.2. Estruturas Condicionais

- **`if / else if / else`:** Decisões lógicas de múltiplos caminhos (ex: validação de status de pedidos).
- **Operador Ternário (`condicao ? valor_se_vazio : valor_se_falso`):**
  `php $operador = ($tipo === 'Entrada') ? '+' : '-';`
  Onde encontrar: `api_estoque_v2.php`.

- **Null Coalescing Operator (`??`):** Fornece um valor padrão caso a variável seja nula ou indefinida: `php $duracao_dias = (isset($_POST['duracao_dias']) && $_POST['duracao_dias'] !== '') ? (int)$_POST['duracao_dias'] : null;`

1.3. Estruturas de Repetição (Laços / Loops)

- **`foreach`:** Iteração sobre coleções e listas associativas em PHP e JavaScript:
- PHP: Percorrer assinantes para geração de pedidos em lote em `api_admin_gerar_pedidos_v2.php`.
- JavaScript: Renderizar cards de produtos no DOM em `catalogo.js`.
- **`while`:** Consumo de cursores de banco de dados (`while ($row = $stmt->fetch())`).

1.4. Tratamento de Exceções (`try / catch / finally / throw`)

- **Conceito:** Mecanismo estruturado para capturar erros em tempo de execução sem derrubar o servidor.
- **No Projeto:**
- Todas as APIs envolvem as operações críticas em blocos `try { ... } catch (PDOException $e) { ... } catch (Exception $e) { ... }`.
- Disparo de erros controlados: `throw new Exception("Saldo insuficiente.");`.
- Reversão automática de transações ACID no `catch` via `$pdo->rollBack()`.
  Onde encontrar: `Pedido.php`, `api_checkout_v2.php`.

---

2. Vetores, Matrizes e Estruturas de Dados (Arrays & Dicionários)

2.1. Tipos de Vetores

- **Arrays Indexados (Numéricos):** Lista linear de elementos ordenada por índices de `0` a `N-1`.
- Exemplo em JS: `['Temporários', 'Legumes', 'Verduras', 'Frutas']` em `catalogo.js`.
- **Arrays Associativos (Dicionários / Mapas Chave-Valor):** Coleções indexadas por strings textuais.
- Exemplo em PHP: `['id' => 10, 'nome' => 'Tomate Orgânico', 'preco' => 8.50]` em `Produto.php`.
- Exemplo em JS: Mapeamento de ícones por categoria `icones['Legumes'] = '🍅'`.
- **Matrizes / Arrays Multidimensionais:** Vetores dentro de vetores.
- Exemplo: Lista de itens do carrinho contendo atributos aninhados de produto, peso e preço.
- Exemplo no Mapa: Coordenadas geográficas `[[lat1, lng1], [lat2, lng2]]` para ajuste de zoom no Leaflet (`entregador.js`).

2.2. Funções de Alta Ordem e Métodos Funcionais em Arrays

Em PHP:

Função	O que faz?	Onde está aplicada?
`array_map()`	Transforma cada item de um array aplicando uma função de callback.	Sanitização de IDs numéricos em `api_admin_gerar_pedidos_v2.php`.
`array_column()`	Extrai uma coluna específica de uma matriz de registros do banco.	Obtenção de lista de IDs de pedidos em `api_admin_editar_kit_semana.php`.
`in_array()`	Verifica se um elemento existe dentro de um vetor (busca linear).	Verificação de permissões RBAC em `cors.php` e checagem de trocas fixas.
`implode()`	Junta elementos de um vetor em uma única string com delimitador.	Construção de placeholders SQL `?, ?, ?` para cláusulas `WHERE IN (...)`.
`explode()`	Quebra uma string em um vetor a partir de um separador.	Extração de nomes compostos ou itens de preferências.

Em JavaScript:

Método	O que faz?	Onde está aplicada?
`filter()`	Cria um novo array apenas com os elementos que atendem a um critério.	Filtros de categorias na vitrine da home em `index.js`.
`map()`	Mapeia cada objeto do array transformando-o em marcação HTML.	Geração de linhas de entrega no app do motorista em `entregador.js`.
`reduce()`	Reduz todo o vetor a um único valor escalar acumulado.	Cálculo do subtotal e do peso total em kg da cesta em `carrinho.js`.
`find()`	Retorna o primeiro elemento que satisfaz a condição de busca.	Localização de item por ID no carrinho para incremento de quantidade.
`splice()`	Remove ou substitui elementos de um array pelo índice.	Exclusão de itens do carrinho quando a quantidade é zerada.
Spread Operator (`...`)	Expande os elementos de um vetor (clonagem superficial).	Algoritmo de roteirização gulosa em `admin.js`.

3. Programação Orientada a Objetos (POO)

3.1. Pilares da POO no Projeto

```
classDiagram
    class Database {
        -PDO instancia$
        +getConexao()$ PDO
    }
    class Usuario {
        -PDO pdo
        +buscarPorEmail(email)
        +cadastrar(...)
    }
    class Pedido {
        -PDO pdo
        +processarCarrinho(usuarioId, itens)
        +atualizarStatus(id, tipo, valor)
    }
    class MercadoPagoService {
        -string accessToken
        +criarPagamento(dados, idempotencyKey)
    }
    Usuario --> Database : Obtém Conexão
    Pedido --> Database : Obtém Conexão
    Pedido ..> MercadoPagoService : Liquidação
```

1. **Classes e Instanciação:**
2. As classes representam modelos do domínio de negócio (ex: `class Pedido`, `class Usuario`).
3. Instanciação de objetos: `$pedidoModel = new Pedido();`.
4. **Encapsulamento e Modificadores de Acesso:**
5. Atributos privados (`private $pdo;`) garantem que o estado interno do objeto não seja corrompido de fora.
6. Métodos públicos (`public function processarCarrinho(...)`) expõem a interface limpa da classe.
7. **Métodos e Atributos Estáticos (`static`):**
8. Métodos utilitários que pertencem à classe e não a uma instância individual:
   - `Database::getConexao()` em `Database.php`.
   - `Security::checkCSRF()` e `Security::checkRateLimit()` em `Security.php`.
   - `Validator::validarCPF()` em `Validator.php`.
9. **Construtores (`__construct`):**
10. Inicializam dependências fundamentais na instanciação do objeto:
    `php public function __construct() { $this->pdo = Database::getConexao(); }`

4. Padrões de Projeto (Design Patterns) e Arquitetura

4.1. Singleton (Criação)

- **Objetivo:** Garantir que uma classe tenha apenas **uma única instância** globalmente e fornecer um ponto de acesso global a ela.
- **Aplicação:** `Database.php`. Evita abrir dezenas de conexões TCP simultâneas com o MySQL durante uma única requisição HTTP.

4.2. Data Mapper / Model Layer (Estrutural)

- **Objetivo:** Separar as regras de negócio em memória do esquema de tabelas do banco de dados relacional.
- **Aplicação:** Todas as classes dentro de `versao_estudo/app/Models/` (`Cliente.php`, `Produto.php`, `Pedido.php`). As APIs chamam métodos expressivos (`$clienteModel->listarTodos()`) em vez de escrever SQL diretamente nos controladores.

4.3. Gateway / Service Layer (Comportamental)

- **Objetivo:** Isolar e encapsular a comunicação com uma API externa de terceiros.
- **Aplicação:** `MercadoPagoService.php`. Centraliza as chamadas cURL ao Mercado Pago, serialização JSON e envio do header de idempotência (`X-Idempotency-Key`).

4.4. Interceptor / Monkey Patching (Frontend)

- **Objetivo:** Interceptar chamadas nativas de rede para adicionar comportamento transversal (segurança, headers, credenciais) de forma transparente.
- **Aplicação:** `utils.js`. Sobrescreve `window.fetch` para obter o token CSRF e anexá-lo automaticamente a todas as requisições `POST/PUT/DELETE`.

---

5. Banco de Dados Relacional e Linguagem SQL

5.1. Modelagem Relacional (DER)

- **Entidades:** `usuarios`, `enderecos`, `assinaturas`, `produtos`, `pedidos`, `itens_pedido`, `faturas`, `preferencias`, `movimentacoes_estoque`, `rate_limits`.
- **Relacionamentos:**
- `1 : N` (Um usuário tem muitos pedidos; um pedido tem muitos itens).
- `1 : 1` (Um usuário tem uma assinatura ativa).

5.2. Comandos e Cláusulas SQL Utilizadas

Comando / Cláusula	Objetivo	Exemplo no Código
`INNER JOIN`	Relaciona tabelas trazendo apenas correspondências exatas em ambas.	Obter nome do cliente e dados do pedido em `api_admin_pedidos_v2.php`.
`LEFT JOIN`	Mantém todos os registros da tabela à esquerda, mesmo sem correspondência na direita.	Trazer clientes mesmo que nunca tenham feito nenhum pedido em `Cliente.php`.
`GROUP BY`	Agrupa linhas com valores iguais para cálculos agregados.	Consolidar a demanda total de cada hortaliça por produto em `api_admin_relatorio_compras.php`.
`SUM()` / `COUNT()`	Calcula a soma total ou contagem de registros.	Somar o faturamento mensal total e total de assinantes no `Dashboard.php`.
`GROUP_CONCAT()`	Concatena valores de múltiplas linhas agrupadas em uma única string delimitada por vírgula.	Agregar todas as restrições alimentares de um cliente em uma coluna em `Cliente.php`.
`COALESCE()`	Retorna o primeiro valor não nulo de uma lista de argumentos.	Fallback entre endereço cadastral e endereço na tabela de endereços em `api_logistica_v2.php`.
`YEARWEEK()`	Agrupa datas por ano e semana epidemiológica/ calendário (0 a 53).	Filtrar apenas os pedidos pertencentes ao ciclo da semana atual (`YEARWEEK(data_pedido, 0) = YEARWEEK(NOW(), 0)`).

5.3. Transações ACID

- **Atomicidade & Consistência:** Quando um pedido é processado, deve-se:
- Inserir registro na tabela `pedidos`.
- Inserir múltiplos registros em `itens_pedido`.
- Abater estoque na tabela `produtos`.
- Abater créditos na tabela `usuarios`.
  Se qualquer um dos 4 passos falhar, **nenhuma alteração é persistida** (`$pdo->rollBack()`).

---

6. Algoritmos e Lógica Aplicada

6.1. Algoritmo do Módulo 11 (Validação de CPF)

- **Conceito:** Cálculo ponderado dos dois dígitos verificadores (DV1 e DV2) do Cadastro de Pessoa Física brasileiro.

- **Complexidade:** $O(1)$ (tamanho fixo de 11 dígitos).
- **Implementação:**
- PHP: `Validator.php`.
- JavaScript: `validarCPF()` em `utils.js`.

6.2. Algoritmo do Vizinho Mais Próximo (Roteirização Logística)

- **Conceito:** Algoritmo guloso (*Greedy Heuristic*) para resolver o problema do caixeiro-viajante simplificado.
- **Funcionamento:**
- Inicia nas coordenadas da sede/galpão da Faz Bem (Uruguaiana/RS: `-29.7603, -57.0811`).
- Calcula a distância euclidiana para todos os endereços não visitados.
- Seleciona o cliente com menor distância, adiciona-o à rota e move a posição atual para esse ponto.
- Repete até que todas as entregas estejam ordenadas sequencialmente.
- **Onde encontrar:** Função `autoOrdenarRotas()` em `admin.js`.

```
function calcularDistanciaLocal(lat1, lon1, lat2, lon2) {
    const dx = lat1 - lat2;
    const dy = lon1 - lon2;
    return Math.sqrt(dx * dx + dy * dy);
}
```

6.3. Algoritmo de Conversão de Unidades e Pesagem Fracionada

- **Problema:** Hortifrútis são vendidos por quilo (kg), mas clientes compram por unidades (ex: 3 tomates) ou frações (ex: 250g).
- **Lógica de Conversão:**
- Se a unidade no banco é `kg` e o cliente escolhe `un`, o sistema multiplica pelo peso médio cadastrado (`peso_estimado_g`) e divide por 1000.
- Na pesagem do galpão, a balança fornece a `quantidade_real` e o sistema recalcula o `preco_real = quantidade_real * preco_unitario`.
  Onde encontrar: `Producao.php`, `Pedido.php` e `separador.js`.

---

7. Desenvolvimento Web, Protocolo HTTP e APIs RESTful

7.1. Anatomia da Comunicação HTTP

- **Métodos Utilizados:**
- `GET`: Consulta idempotente de recursos (catálogo, histórico, rotas).
- `POST`: Criação e mutação de recursos com payload JSON/Multipart (login, cadastro, checkout).
- `DELETE`: Exclusão lógica de recursos (remover produto, excluir funcionário).
- **Códigos de Resposta (Status Codes):**
- `200 OK`: Sucesso na requisição.

- `400 Bad Request`: Parâmetros inválidos ou incompletos enviados pelo cliente.
- `403 Forbidden`: Acesso não autenticado ou sem permissão de papel (RBAC).
- `405 Method Not Allowed`: Verbo HTTP incompatível com a rota.
- `429 Too Many Requests`: Limite de taxa de requisições excedido (Rate Limit).
- `500 Internal Server Error`: Erro inesperado ou falha de banco de dados no servidor.

7.2. Manipulação do DOM e Estado no Cliente

- **Assincronismo:** Uso intensivo de `fetch()`, `async/await` e `Promise` para comunicação em background sem recarregamento de página.
- **Persistência Client-Side:** Uso de `localStorage.getItem('fazbem_carrinho')` e `localStorage.setItem(...)` para manter o carrinho sincronizado entre abas e páginas.

---

8. Segurança da Informação e Diretrizes OWASP

8.1. OWASP Top 10 Implementado

Vulnerabilidade	Risco Mitigado	Implementação Técnica no Código
A01: Broken Access Control	Usuários acessando painéis administrativos ou dados de outros clientes.	Verificação rigorosa de `$_SESSION['tipo_usuario']` em todas as APIs administrativas e filtros por `usuario_id` da sessão em endpoints de clientes (`cors.php`).
A02: Cryptographic Failures	Vazamento de senhas em caso de invasão da base de dados.	Hashing unidirecional com algoritmo Bcrypt via `password_hash($senha, PASSWORD_DEFAULT)` em `Usuario.php` .
A03: Injection (SQL Injection)	Injeção de comandos SQL maliciosos via inputs de formulários.	Desativação de emulação no PDO (`ATTR_EMULATE_PREPARES => false`) e uso 100% obrigatório de consultas parametrizadas com placeholders `?` (`Database.php`).
A04: Insecure Design (CSRF)	Sites de terceiros forçando requisições em nome do usuário autenticado.	Geração de tokens criptográficos pseudoaleatórios (`bin2hex(random_bytes(32))`) validados em tempo constante com `hash_equals()` via `Security.php` .
A07: Identification and Auth Failures	Ataques de força bruta automatizados contra a tela de login.	Algoritmo de Rate Limiting persistido na tabela `rate_limits` do MySQL com bloqueio temporário após exceder limite por IP (`Security.php`).
Stored & Reflected XSS	Injeção de scripts JavaScript através de campos de texto ou cadastro.	Função global `escapeHTML()` em `utils.js` que substitui caracteres perigosos (`<` , `>` , `&` , `"` , `'`) antes da inserção no DOM.

---

9. Integrações Externas, Webhooks e APIs de Terceiros

9.1. Mercado Pago (Checkout Transparente & Bricks)

- **Payment Brick (Frontend):** SDK JavaScript que renderiza os campos de cartão de crédito e gera chaves Pix com segurança sem que os dados de cartão toquem no servidor local (`perfil.js`).
- **cURL Service (Backend):** Comunicação segura com os servidores do Mercado Pago via cURL em `MercadoPagoService.php` .
- **Webhook Assíncrono (IPN):** Endpoint `api_mp_webhook.php` que recebe notificações push do Mercado Pago, valida a assinatura criptográfica HMAC SHA256 e dá baixa automática nas faturas quitadas.

9.2. Mapas e Serviços Geoespaciais

- **Leaflet.js:** Biblioteca open-source para renderização de mapas interativos, marcadores arrastáveis e polylines de rota (`cadastro.js` , `entregador.js`).
- **Nominatim API (OpenStreetMap):** Geocodificação de endereços em tempo real convertendo logradouros em latitude/longitude.
- **OSRM (Open Source Routing Machine):** Cálculo de malha viária e rotas de trânsito em formato GeoJSON.

---

10. Frontend Moderno, UI, CSS e Responsividade

10.1. Variáveis Customizadas (CSS Custom Properties)

- Definição de tokens de cores, espaçamentos e sombras centralizados no `:root` :
  `css :root { --primary: #166534; --primary-light: #22c55e; --bg-body: #f8fafc; --border: #e2e8f0; --text-main: #0f172a; --text-muted: #64748b; }`
  Onde encontrar: `index.css` , `admin.css` .

10.2. CSS Grid e Flexbox

- **Flexbox (Unidimensional):** Alinhamento de cabeçalhos, barras de ações, botões e controles de quantidade (`display: flex; justify-content: space-between; align-items: center;`).
- **CSS Grid (Bidimensional):** Grades de produtos responsivas com dimensionamento dinâmico sem quebras:
  `css .product-grid { display: grid; grid-template-columns: repeat(auto-fill, minmax(240px, 1fr)); gap: 20px; }`

10.3. Design Responsivo e Mobile-First

- Uso de **Media Queries** (`@media (max-width: 768px)`) para adaptar o menu lateral em drawer deslizante, aumentar a área de toque em telas mobile e reorganizar tabelas em cartões.
- **Folhas de Impressão:** Regras `@media print` e classes `.no-print` para gerar folhas de separação limpas em papel para o galpão.

🎯 Roteiro de Autoavaliação e Exercícios Práticos

Para fixar os conteúdos estudados, tente responder e resolver os seguintes desafios no código:

1. **Vetores e Lógica:** No arquivo `api_admin_relatorio_compras.php`, como os dados brutos vindos do SQL são transformados em um vetor de strings legíveis como `"15 maços"` ou `"4,5 kg"`?
2. **POO e Singleton:** Por que o construtor da classe `Database.php` é definido como `private`? O que aconteceria se tentássemos instanciar `$db = new Database();` diretamente?
3. **Segurança e CSRF:** Explique o fluxo completo de um token CSRF desde a sua emissão em `api_csrf.php`, passando pela interceptação em `utils.js`, até a validação em `Security.php`.
4. **SQL e Agregações:** Na classe `Cliente.php`, qual a função do `LEFT JOIN` combinado com `GROUP_CONCAT` na listagem de clientes?
5. **Algoritmos:** No arquivo `admin.js`, explique como a função `autoOrdenarRotas()` calcula a ordem das entregas com base no algoritmo do vizinho mais próximo.